

# STUDENT ACADEMIC PORTAL SYSTEM

---

A Comprehensive Project Report for Principles of Structured Programming

## CHAPTER 1: INTRODUCTION

The Student Academic Portal System is a desktop-based application developed using Python and Tkinter.

The purpose of the system is to improve the management of student academic records in educational institutions.

Many schools still rely on paper-based systems which are prone to loss, duplication, and errors.

This project provides a computerized solution for storing, retrieving, updating, and reporting student information.

### 1.1 Background of the Study

Educational institutions generate large amounts of student information every academic year.

Managing these records manually is time-consuming and inefficient. The introduction of information technology into education has made it possible to automate these processes. This project demonstrates how structured programming can be used to build an academic management system.

### 1.2 Problem Statement

Manual record management causes delays, inaccurate calculations, difficulty in retrieving information, and increased risk of losing important academic records. The proposed Student

Academic Portal System addresses these challenges through automation.

### **1.3 Aim of the Project**

To design and implement a Student Academic Portal System that simplifies academic record management.

### **1.4 Objectives**

- Register students
- Store academic records
- Search student information
- Update records
- Delete records
- Calculate GPA
- Generate reports
- Improve efficiency and accuracy

### **1.5 Significance of the Study**

The project benefits students, lecturers, administrators, and institutions by providing fast access to records, reducing paperwork, and improving academic management.

## **CHAPTER 2: SYSTEM ANALYSIS**

System analysis involves studying the existing system and identifying weaknesses and opportunities for improvement.

## **2.1 Existing System**

Most institutions maintain records manually through files, notebooks, and spreadsheets.

This approach can lead to misplaced records, delays, and inaccuracies.

## **2.2 Proposed System**

The proposed system provides a graphical user interface where administrators can manage student information, calculate GPA, and generate reports automatically.

## **2.3 Advantages of the Proposed System**

- Faster record retrieval
- Better data accuracy
- Improved security
- Reduced paperwork
- Automated report generation

## **CHAPTER 3: SYSTEM DESIGN**

The design phase defines the structure and operation of the application.

### **3.1 Input Design**

The system accepts Student ID, Name, Gender, Department, Level, Course, and Score.

## **3.2 Process Design**

The system validates data, stores records, calculates GPA, determines pass/fail status, and generates reports.

## **3.3 Output Design**

Outputs include student records, GPA results, pass/fail status, and academic reports.

## **3.4 Flow of the System**

Start → Login → Dashboard → Add/Search/Update/Delete Student → Generate Reports → Exit

# **CHAPTER 4: IMPLEMENTATION**

Implementation involves translating the design into a working application using Python.

## **4.1 Programming Language**

Python was selected because it is easy to learn, powerful, and supports GUI development through Tkinter.

## **4.2 GUI Development**

Tkinter was used to design the graphical interface. The GUI contains forms, buttons, labels, entry fields, and report sections.

## **4.3 File Storage**

Student records are stored using JSON files. This allows data to remain available even after the program is closed.

## **4.4 Modules Developed**

1. Login Module
2. Student Registration Module
3. Search Module
4. Update Module
5. Delete Module
6. GPA Calculator
7. Report Generator

# **CHAPTER 5: STRUCTURED PROGRAMMING**

## **CONCEPTS USED**

This project demonstrates the principles taught in structured programming.

## **5.1 Functions**

Functions were used to divide the system into smaller manageable units such as `add_student()`, `search_student()`, `update_student()`, `delete_student()`, and `generate_report()`.

## **5.2 Conditional Statements**

If-else statements were used for login validation, GPA calculation, and pass/fail determination.

## **5.3 Loops**

Loops were used to iterate through student records when searching, displaying, and generating reports.

## **5.4 Data Structures**

Lists and dictionaries were used to store and organize student information efficiently.

## **5.5 Exception Handling**

Try-except blocks were used to prevent program crashes when files are missing or invalid data is entered.

# **CHAPTER 6: TESTING AND RESULTS**

Testing was conducted to ensure the system functions correctly.

## **Test Case 1**

Add a new student and verify the record appears in the system.

## **Test Case 2**

Search for an existing student and verify accurate retrieval.

## **Test Case 3**

Update student information and verify changes are saved.

## **Test Case 4**

Delete a student record and verify removal from storage.

## **Test Case 5**

Generate a report and verify all student information is included.

## **CHAPTER 7: SDG ALIGNMENT**

The project aligns with Sustainable Development Goal 4 (Quality Education). It promotes digital transformation in education and improves academic administration.

## **CHAPTER 8: CONCLUSION**

The Student Academic Portal System successfully demonstrates how structured programming can be applied to solve real-world educational challenges. The project improves efficiency, accuracy, and accessibility of academic records.

## **CHAPTER 9: RECOMMENDATIONS**

Future improvements may include database integration, cloud storage, mobile access, attendance tracking, course registration, biometric login, and transcript generation.

## **OUR PROJECT PRESENTATION GUIDE**

During our presentation:

1. We introduce the problem.
2. Explain the objectives.
3. Discuss technologies used.
4. Demonstrate the login system.
5. Demonstrate adding, updating, searching, and deleting students.
6. Show GPA calculation.
7. Generate a report.

8. Explain SDG alignment.
9. Present conclusions and recommendations.
10. Answer questions confidently.